МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»
ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ

Лабораторна робота №3

з дисципліни “Теорія інформації і кодування”

на тему: “ СТИСНЕННЯ ДАНИХ В ДИСКРЕТНИХ ПОВІДОМЛЕННЯХ З ВИКОРИСТАННЯМ АЛГОРИТМУ ГАФФМАНА.”

Виконав:
студент III курсу
групи ДА-11
Сидько Нікіта Сергійович

Київ-2023

Мета роботи вивчення алгоритмів ефективного кодування без втрат та їх реалізації для стиснення даних в дискретних повідомленнях з використанням алгоритму Гаффмана, оцінка надмірності повідомлень, розробка програми для дослідження ефективності кодування даних у файлах різних типів і довільної довжини.

Завдання

1. Ознайомитися з методикою ефективного кодування Гаффмана.

2. Використовуючи табличний процесор MS Excel закодуйте алгоритмом Гаффмана текстове повідомлення з вашим прізвищем ім'ям, по батькові, датою і місцем народження. Підрахуйте ентропію текстового повідомлення, середню довжину кодових комбінацій закодованого повідомлення, коефіцієнт стиснення і оцініть ефективність кодування. Перевірте префіксність коду і розкодуйте повідомлення.

3. Розробити програму на мові програмування високого рівня для кодування і декодування текстових файлів довільної довжини. При вводі текстових даних передбачити два режиму вводу кодів: введення з клавіатури, а також введення даних з текстового файлу з використанням однобайтових кодових таблиць. За допомогою розробленої програми перевірити дані, отримані при виконанні п.2.

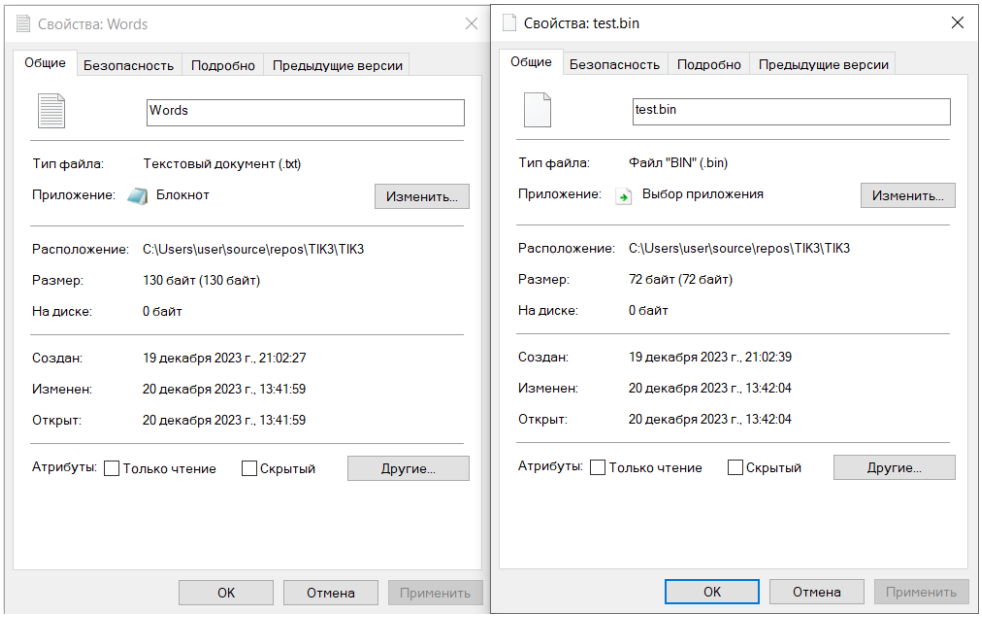
4. За допомогою розробленої при виконанні п.3 програми дослідити залежність коефіцієнту стиснення від довжини текстового файлу. Результати представити в табличній формі.

5.Модіфікувати програму, розроблену при виконанні п.4, для стиснення файлів довільних типів даних за алгоритмом Гаффмана. Провести дослідження ефективності розробленої програми шляхом порівняння результатів стиснення текстових, мультимедійних і exe –файлів з допомогою розробленої програми і архіватора WinZip . Результати занести в таблицю

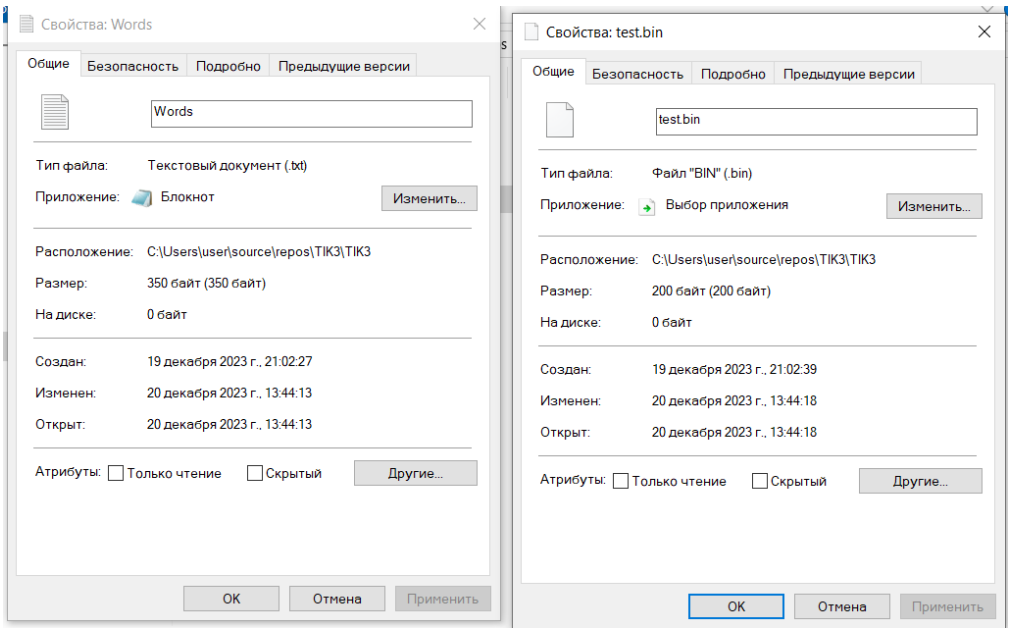
Хід роботи

```
Дерево Гаффмана
Символ  Код
2       0000
.       0001
н       00100
р       00101
в       0011
5       0100
ї       01010
а       01011
ч       01100
й       01101
с       0111
        100
0       1010
к       1011
і       1100
и       1101
о       11100
г       111010
д       111011
4       111100
е       111101
т       111110
ь       111111
```

Введемо ще декілька повідомлень для обрахунку коефіцієнта стиснення:

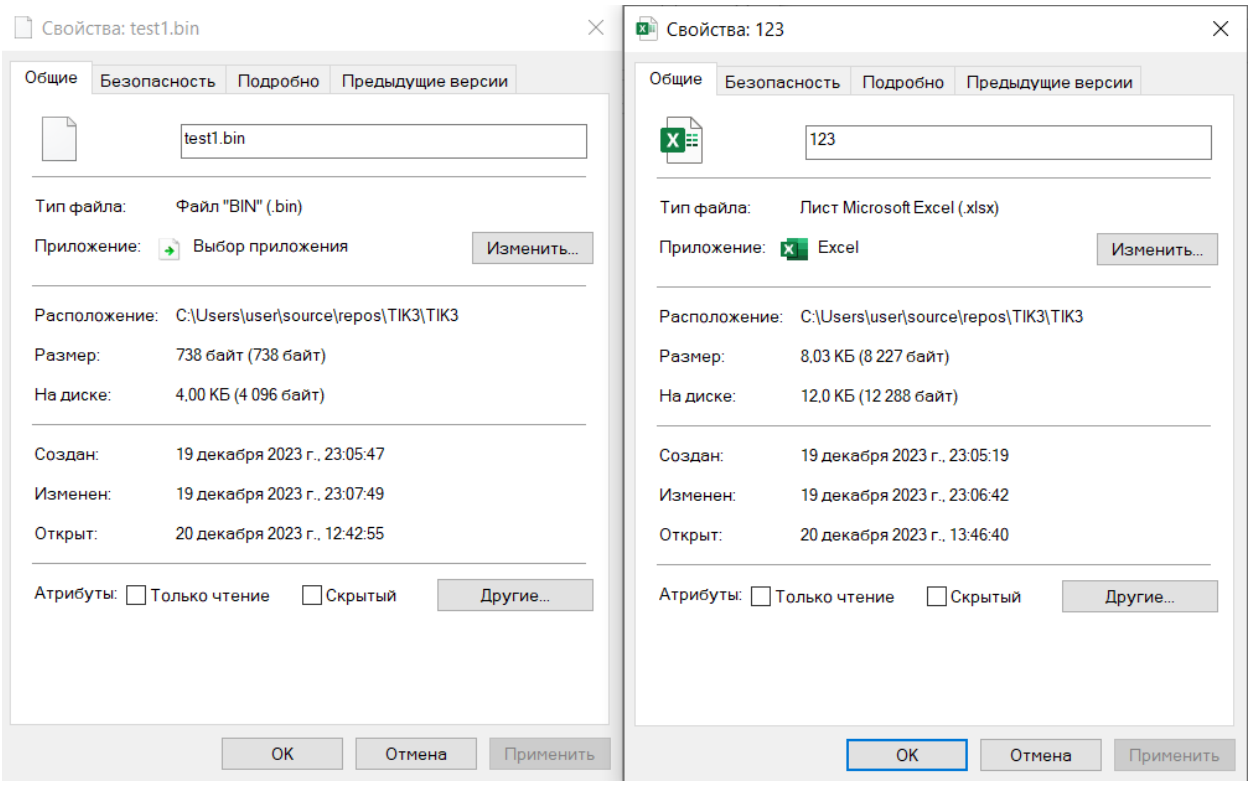


Коеф стиснення = 1.805



Коеф стиснення = 1.75

Пропишемо в коді, наприклад для File.xlsx, та протестуємо:



Отже дана програма може стискати довільні типи даних.

Вихідний файл	Програма	Архіватор
Назва: Words1.txt **Розмір**: 130	**Розмір: 72** **Коеф :** 1.805	**Розмір:** 87 **Коеф :** 1.49
Назва: Words1.txt **Розмір**: 350	**Розмір**: **200** **Коеф:** 1.75	**Розмір:** 219 **Коеф:** 1.58

Висновок.

У ході виконання 3тьої лабораторної роботи, була розроблена програма, для стиснення файлу методом Хаффмана. Для прикладу був взятий текстовий документ у якому було написано ПІБ, текст був закодований на записаний у бітах в BIN файл, після чого файл був побітово зчитаний та розкодований

Лістинг коду

```
#include <iostream>
#include <fstream>
#include <map>
#include <cmath>
#include <vector>
#include <algorithm>
#include <string>
#include <queue>
#include "Windows.h"
#include <bitset>
#include <sstream>

void setConsoleEncoding() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
}

struct Symbol {
    char symbol;
    int frequency;
    std::string code;
};

struct Node {
    Symbol symbol;
    Node* left;
    Node* right;
};

bool compareProbability(const Node* a, const Node* b) {
    return a->symbol.frequency > b->symbol.frequency;
}
```

```
void calculateValues(const std::map<char, int>& charCount, int totalCharacters) {
    float PiSUM = 0;
    float LogSUM = 0;
    float EntrSUM = 0;

    std::cout << "Таблиця:" << std::endl;
    std::cout << "Символ\tЧастота\tP[i]\t\tLog(P[i])\tEntropy" << std::endl;

    for (const auto& pair : charCount) {
        PiSUM += static_cast<float>(pair.second) / totalCharacters;
        LogSUM += log2(static_cast<float>(pair.second) / totalCharacters);
        EntrSUM += static_cast<float>(pair.second) / totalCharacters *
log2(static_cast<float>(pair.second) / totalCharacters);

        if (pair.first == ' ') {
            std::cout << "Space\t";
        }
        else {
            std::cout << pair.first << "\t";
        }

        std::cout << pair.second << "\t" << static_cast<float>(pair.second) /
totalCharacters
            << "\t\t" << log2(static_cast<float>(pair.second) / totalCharacters)
            << "\t" << -1 * static_cast<float>(pair.second) / totalCharacters *
log2(static_cast<float>(pair.second) / totalCharacters)
            << std::endl;
    }

    EntrSUM *= -1;

    std::cout << std::endl;
    std::cout << "Symbol count: " << totalCharacters << std::endl;
    std::cout << "Sum P[i]: " << PiSUM << std::endl;
    std::cout << "Sum Log(P[i]): " << LogSUM << std::endl;
    std::cout << "H: " << EntrSUM << std::endl;
    std::cout << std::endl;

    std::cout << "Amount of info: " << totalCharacters * EntrSUM << std::endl;
    std::cout << "Max entropy: " << log2(charCount.size()) << std::endl;
    std::cout << "Absolute excess: " << log2(charCount.size()) - EntrSUM <<
std::endl;
    std::cout << "Relative excess: " << (log2(charCount.size()) - EntrSUM) /
EntrSUM << std::endl;
    std::cout << std::endl;
}

std::map<char, int> readCharacterCounts(const std::string& filename, int&
totalCharacters) {
    std::map<char, int> charCount;
    uint8_t character;

    std::ifstream inputFile(filename, std::ios::binary);

    if (!inputFile.is_open()) {
        std::cerr << "Unable to open file." << std::endl;
        exit(1);
    }

    while (true) {
        character = inputFile.get();
        if (inputFile.eof()) {
            break;
        }
        charCount[character]++;
```

```
            totalCharacters++;
        }

        inputFile.close();

        return charCount;
    }

    void sortTable(const std::map<char, int>& charCount, std::vector<Node*>&
    probabilityVector, int totalCharacters) {
        for (const auto& pair : charCount) {
            Node* newNode = new Node;
            newNode->symbol = { pair.first, pair.second, "" };
            newNode->left = nullptr;
            newNode->right = nullptr;
            probabilityVector.push_back(newNode);
        }

        std::sort(probabilityVector.begin(), probabilityVector.end(),
    compareProbability);
    }

    void printTable(const std::vector<Node*>& probabilityVector, const std::map<char,
    int>& charCount) {
        std::cout << "Відсортована таблиця:" << std::endl;
        std::cout << "Символ\tЧастота\tКод" << std::endl;

        for (const auto& node : probabilityVector) {
            const auto& symbol = node->symbol;
            if (symbol.symbol == ' ') {
                std::cout << "Space\t";
            }
            else {
                std::cout << symbol.symbol << "\t";
            }
            std::cout << symbol.frequency << "\t" << symbol.code << std::endl;
        }
    }

    void buildHuffmanTree(std::vector<Node*>& probabilityVector, Node*& root) {
        std::priority_queue<Node*, std::vector<Node*>, decltype(&compareProbability)>
    pq(&compareProbability);
        for (Node* node : probabilityVector) {
            pq.push(node);
        }

        while (pq.size() > 1) {
            Node* left = pq.top();
            pq.pop();
            Node* right = pq.top();
            pq.pop();

            Node* internalNode = new Node;
            internalNode->left = left;
            internalNode->right = right;
            internalNode->symbol.frequency = left->symbol.frequency + right-
    >symbol.frequency;

            pq.push(internalNode);
        }

        root = pq.top();
    }

    void traverseAndPrintTree(Node* root, std::string currentCode) {
```

```
    if (root != nullptr) {
        if (root->left != nullptr) {
            traverseAndPrintTree(root->left, currentCode + "0");
        }

        if (root->right != nullptr) {
            if (root->left == nullptr) {
                std::cout << root->symbol.symbol << "\t" << currentCode <<
std::endl;
            }
            traverseAndPrintTree(root->right, currentCode + "1");
        }

        // Добавлен блок обработки листового узла
        if (root->left == nullptr && root->right == nullptr) {
            std::cout << root->symbol.symbol << "\t" << currentCode << std::endl;
        }
    }
}

void generateHuffmanCodes(Node* root, std::map<char, std::string>& encodingMap) {

    if (root != nullptr) {
        if (root->left != nullptr) {
            root->left->symbol.code = root->symbol.code + "0";
            generateHuffmanCodes(root->left, encodingMap);
        }

        if (root->right != nullptr) {
            root->right->symbol.code = root->symbol.code + "1";
            generateHuffmanCodes(root->right, encodingMap);
        }

        // Добавлен блок обробки листового вузла
        if (root->left == nullptr && root->right == nullptr) {
            encodingMap[root->symbol.symbol] = root->symbol.code;
        }
    }
}

std::string readSentenceFromFile(const std::string& filename) {
    std::ifstream file(filename);

    if (!file.is_open()) {
        std::cerr << "Unable to open file: " << filename << std::endl;
        exit(1);
    }

    std::string sentence;
    std::getline(file, sentence);

    file.close();

    return sentence;
}

std::string encodeSentence(const std::string& sentence, const std::map<char,
std::string>& encodingMap) {
    std::string encodedString;

    for (char character : sentence) {
        auto it = encodingMap.find(character);
        if (it != encodingMap.end()) {
            encodedString += it->second;
        }
```

```
        else {
            std::cerr << "Symbol not found: " << character << std::endl;
        }
    }
    return encodedString;
}

void writeBinaryFile(const std::string& filename, const std::string& data) {
    std::ofstream file(filename, std::ios::binary);
    if (file.is_open()) {
        for (std::size_t i = 0; i < data.length(); i += 8) {
            std::bitset<8> byte;
            for (std::size_t j = 0; j < 8; ++j) {
                if (i + j < data.length()) {
                    byte[7 - j] = (data[i + j] == '1');
                }
            }
            char byteChar = static_cast<char>(byte.to_ulong());
            file.write(&byteChar, 1);
        }
        file.close();
    }
    else {
        std::cerr << "Unable to open file: " << filename << std::endl;
    }
}

std::string reverseString(const std::string& input) {
    std::string reversed;
    for (auto it = input.rbegin(); it != input.rend(); ++it) {
        reversed += *it;
    }
    return reversed;
}

std::string readBinaryFile(const std::string& filename) {
    std::ifstream file(filename, std::ios::binary);
    if (file.is_open()) {
        std::ostringstream oss;
        char byte;

        while (file.read(&byte, 1)) {
            std::bitset<8> bits(byte);
            oss << bits;
        }

        file.close();
        return oss.str();
    }
    else {
        std::cerr << "Unable to open file: " << filename << std::endl;
        return "";
    }
}

void decodeBinaryFile(const std::string& filename, const std::map<std::string,
char>& decodingMap) {

    std::string encodedBits = readBinaryFile(filename);
    std::string currentCode = "";
    std::string decodedSentence = "";
    for (char bit : encodedBits) {
        std::cout << bit;
        //std::cout << currentCode << std::endl;
        currentCode += bit;
```

```
        auto it = decodingMap.find(currentCode);
        if (it != decodingMap.end()) {

            decodedSentence += it->second;
            currentCode.clear();
        }
    }
    std::cout << std::endl;
    std::cout << "Decoded Sentence: " << decodedSentence << std::endl;
}

int main() {

    setConsoleEncoding();

    int totalCharacters = 0;
    std::map<char, int> charCount = readCharacterCounts("Words.txt",
totalCharacters);

    std::vector<Node*> probabilityVector;


    calculateValues(charCount, totalCharacters);
    sortTable(charCount, probabilityVector, totalCharacters);
    printTable(probabilityVector, charCount);

    Node* root = nullptr;
    buildHuffmanTree(probabilityVector, root);

    std::cout << "\nДерево Хаффмана:" << std::endl;
    std::cout << "Символ\tКод" << std::endl;

    traverseAndPrintTree(root, "");

    std::map<char, std::string> encodingMap;
    std::map<std::string, char> decodingMap;

    generateHuffmanCodes(root, encodingMap);


    for ( Node* node : probabilityVector) {
        decodingMap[node->symbol.code] = node->symbol.symbol;
        //decodingMap[element.code] = element.symbol;
    }

    std::string sentence = readSentenceFromFile("Words.txt");
    std::string encodedString = encodeSentence(sentence, encodingMap);

    writeBinaryFile("test.bin", encodedString);
    decodeBinaryFile("test.bin", decodingMap);

    for (Node* node : probabilityVector) {
        delete node;
    }
    delete root;

    return 0;
}
```